Fakultas Informatika
School of Computing
Telkom University

Aplikasi Berbasis Platform

Layout (3) & Navigation

Outline

- Tabview
- Pageview
- Navigation

Tabview

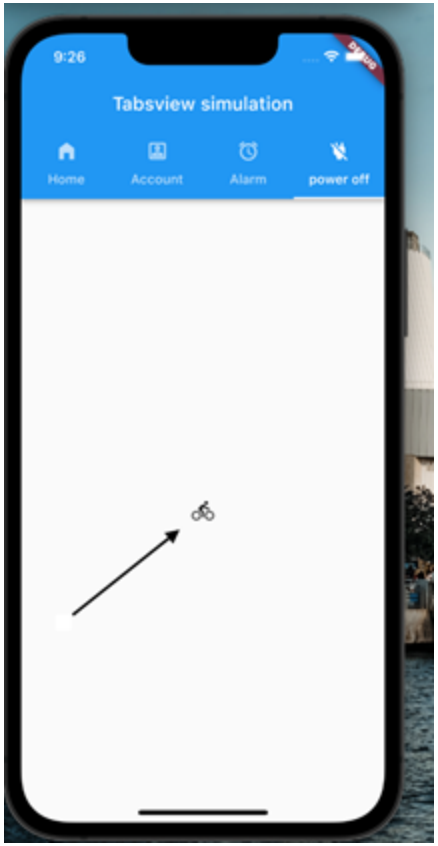
- For the first to create tab is DefaultTabController
- There is 2 option to use the tabview
  - **TabBar:** It is used to display the top view of tabs or more specifically it displays the content of the tab.
  - **TabBarView:** It is used to display the contents when a tab is pressed.

Tabview (Tab Bar)

```
home: DefaultTabController(
    length: 3,
    child: Scaffold(
      appBar: AppBar(
        bottom: const TabBar(
          tabs: [
            Tab(icon: Icon(Icons.home_filled),text: "Home",),
            Tab(icon: Icon(Icons.account_box_outlined),text: "Account",),
            Tab(icon: Icon(Icons.alarm),text: "Alarm",),
          ],
        ),
```

Tabview (TabBarView)

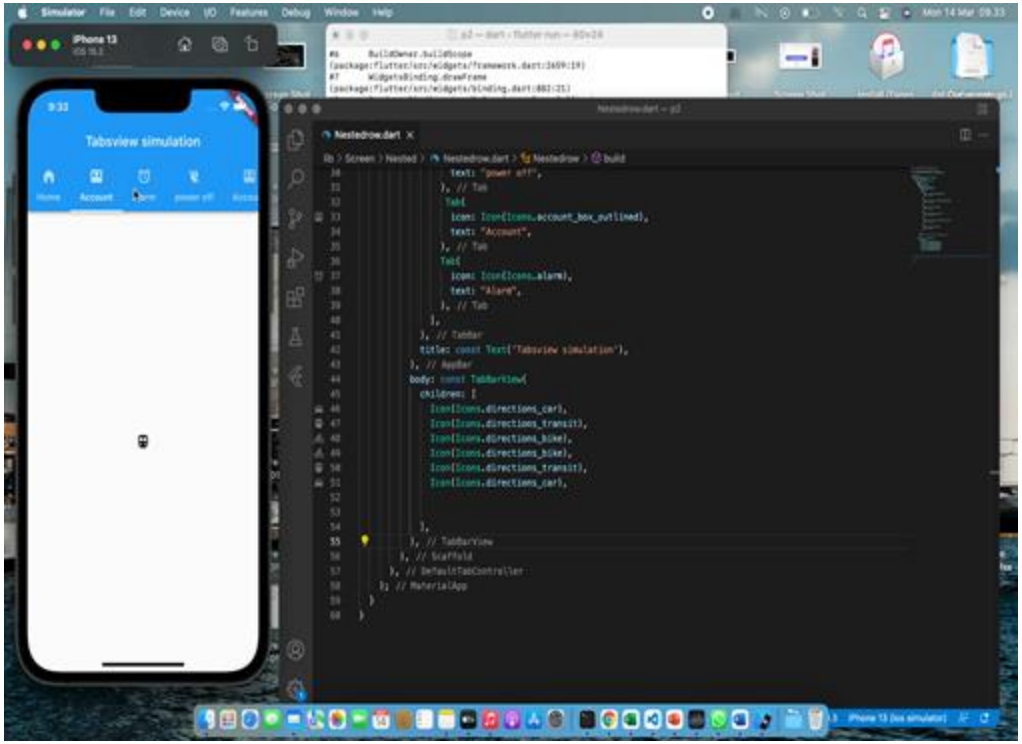
```
body: const TabBarView(
children: [
        Icon(Icons.directions_car),
        Icon(Icons.directions_tran
sit),
        Icon(Icons.directions_bike
),
        Icon(Icons.directions_bike
),
],
```



Tabview (Scrollable)

- There is any altenative for many tabs more than 3
- We can use scrollable for best UX for user

Tabview (Scrollable)




```
home: DefaultTabController(

    length: 3,

    child: Scaffold(

      appBar: AppBar(

        bottom: const TabBar(

          tabs: [

            Tab(icon: Icon(Icons.home_filled),text: "Home",),

            Tab(icon: Icon(Icons.account_box_outlined),text:
"Account",),

            Tab(icon: Icon(Icons.alarm),text: "Alarm",),

          ],

        ),
```

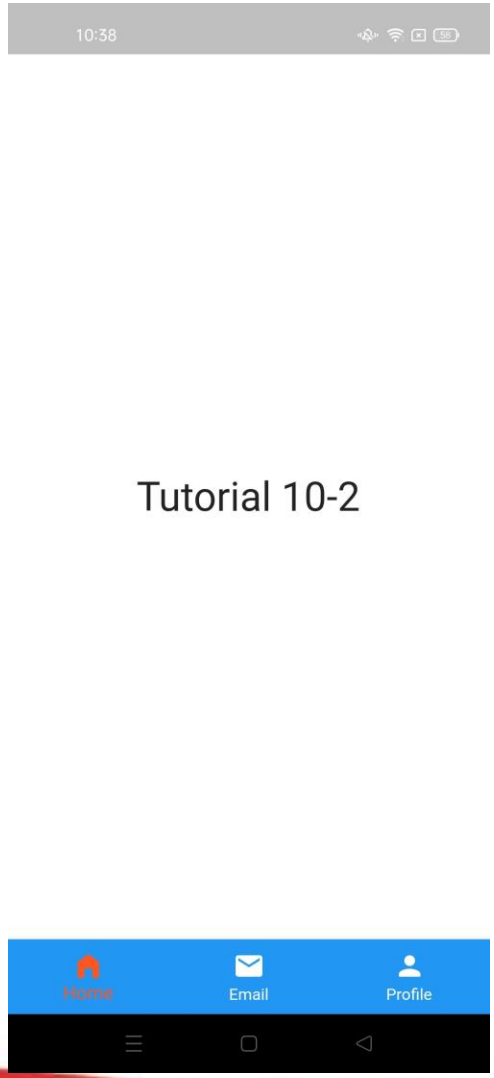
Outline

- TabView
- Pageview
- Navigation

Pageview

- A PageView is a widget which generates scrollable pages on the screen. This can either be a fixed list of pages or a builder function which builds repeating pages. PageView acts similar to a ListView in the sense of constructing elements.
- All types of Page Views can have **horizontal** or **vertically** scrolling pages.
- The types of PageView are:
  - **PageView**
  - **PageView.builder**
  - **PageView.custom**

Pageview

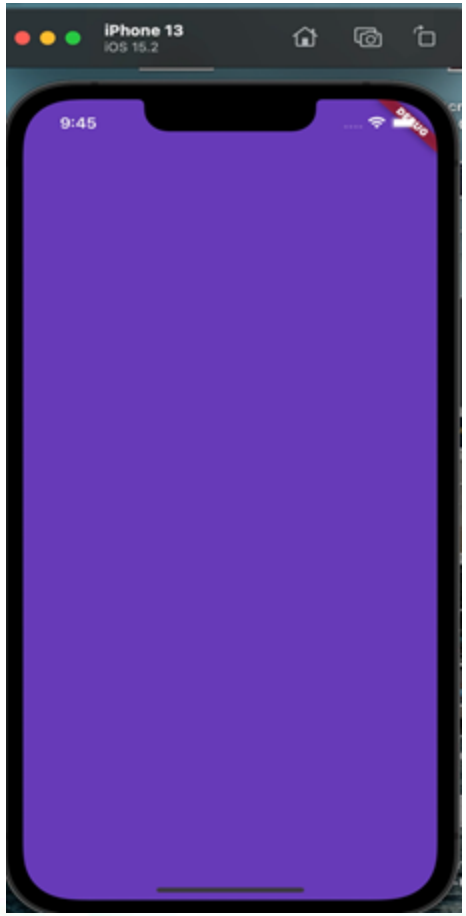


PageView (default Constructor)

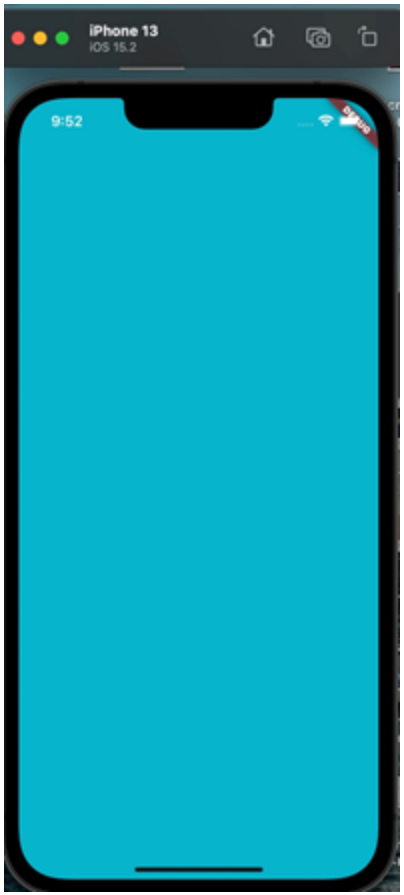
This type takes a fixed list of children (pages) and makes them *scrollable*.

Code :

```
PageView(
  children: <Widget>[
    Container(
      color: Colors.pink,
    ),
    Container(color: Colors.cyan,),
    Container(color: Colors.deepPurple,
    ),],)
```



PageView.builder



Code :

```
PageView.builder(

  itemBuilder: (context, position) {

    return Container(

                color: position % 2 == 0 ? Color.fromARGB(255, 255, 248,
250) :          Color.fromARGB(255, 11, 179, 201),    );

  },

)
```

Outline

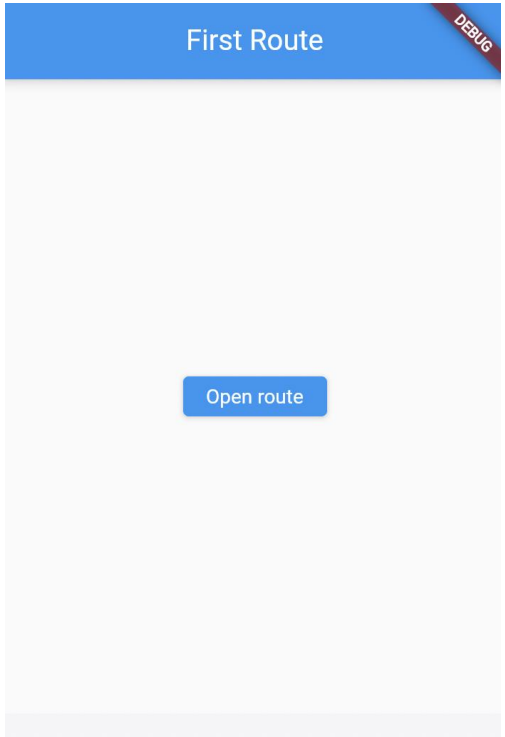
- TabView
- PageView
- Navigation

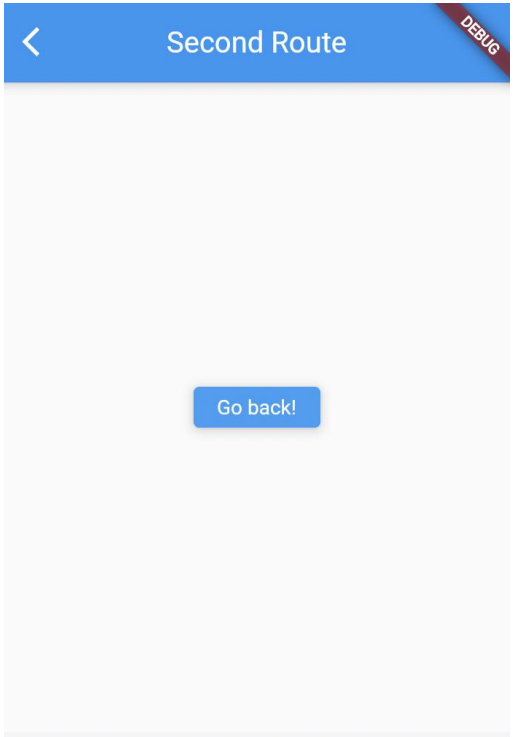
Navigation

- Go to another page when an event occurred
- Variations:
  - Navigate to another screen and go back
  - Navigate with named routes
  - Navigate with passing arguments / data
  - Return data and send data to new screen

Navigate to new screen and go back

```
import 'package:flutter/material.dart';


void main() {

  runApp(const MaterialApp(

    title: 'Navigation Basics',

    home: FirstRoute(),

  ));

}
```





Code Example

```
class FirstRoute extends StatelessWidget {
  const FirstRoute({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('First Route'),
      ),
      body: Center(
        child: ElevatedButton(
          child: const Text('Open route'),
          onPressed: () {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => const
SecondRoute()),
            );
          },
        ),
      ),
    );
  }
}
```

```
class SecondRoute extends StatelessWidget {
  const SecondRoute({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Second Route'),
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.pop(context);
          },
          child: const Text('Go back!'),
        ),
      ),
    );
  }
}
```

Navigate with named routes

```
import 'package:flutter/material.dart';

void main() {
  runApp(
    MaterialApp(
      title: 'Named Routes Demo',
      // Start the app with the "/" named route. In this case, the app starts
      // on the FirstScreen widget.
      initialRoute: '/',
      routes: {
        // When navigating to the "/" route, build the FirstScreen widget.
        '/': (context) => const FirstScreen(),
        // When navigating to the "/second" route, build the SecondScreen widget.
        '/second': (context) => const SecondScreen(),
      },
    ),
  );
}
```

Code Example

```
class FirstScreen extends StatelessWidget {
  const FirstScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('First Screen'),
      ),
      body: Center(
        child: ElevatedButton(
          // Within the `FirstScreen` widget
          onPressed: () {
            // Navigate to the second screen using a named route.
            Navigator.pushNamed(context, '/second');
          },
          child: const Text('Launch screen'),
        ),
      ),
    );
  }
```

```
class SecondScreen extends StatelessWidget {
  const SecondScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Second Screen'),
      ),
      body: Center(
        child: ElevatedButton(
          // Within the SecondScreen widget
          onPressed: () {
            // Navigate back to the first screen by popping the
current route
            // off the stack.
            Navigator.pop(context);
          },
          child: const Text('Go back!'),
        ),
      ),
    );
  }
}
```

Another examples

- Pass arguments : https://docs.flutter.dev/cookbook/navigation/navigate-with-arguments
- Return data and send data to new screen : https://docs.flutter.dev/cookbook/navigation/passing-data

Kuy coding

- Kerjakan Tutorial 3.1 di LMS

Reference

- https://blog.codemagic.io/designing-complex-ui-using-android-constraintlayout/
- https://flutter.dev/
- https://blog.codemagic.io/building-responsive-applications-with-flutter/
- https://medium.com/flutter-community/a-deep-dive-into-pageview-in-flutter-with-custom-transitions-581d9ea6dded

Fakultas Informatika
School of Computing
Telkom University

THANK YOU